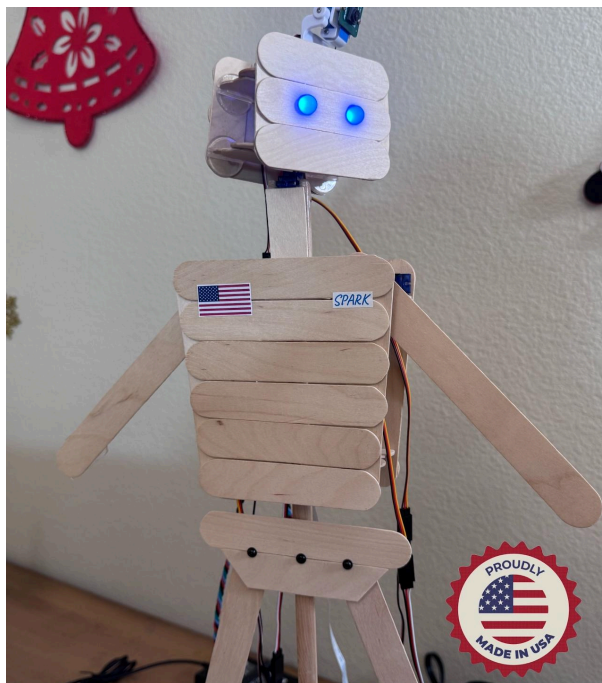




SPARK

Igniting Curiosity Through AI and Robotics

TL;DR.....	2
Introduction.....	2
Identifying a Community Problem and Its Impact.....	2
Alignment with National Priorities and American Innovation.....	2
Technical Design and System Architecture.....	3
Hardware Design and Sensors.....	3
AI Vision Pipeline.....	4
Application Logic and Physical Actions.....	4
Extendability and Additional Applications.....	4
AI Decision Flow.....	4
Conversational AI & Text-to-Speech.....	4
Development Tools and AI Assistance.....	5
Technical Stack Summary.....	5
Design Process, Iteration, and Development.....	5
Process and Development Summary.....	6
AI Integration, Testing, and Validation.....	7
AI Validation Summary.....	8
Key Takeaway.....	9
Originality Through a Reusable AI Platform.....	9
Sharing the Spark with the World.....	9
Additional Notes and Media.....	10
Certification of Originality.....	10

TL;DR

Don't have time to read everything? Simply watch the End-to-End Demo video (3.49 mins) to see SPARK in action, from sensing and AI decision-making to robotic responses, all in one concise overview.



Introduction

My project is called **SPARK**, an AI-powered humanoid robot designed to address real community problems while inspiring students, especially girls, to learn artificial intelligence and robotics through hands-on projects.

SPARK stands for **STEM Projects with AI and Robotics for Kids**, which is also the name of the Girls STEM club that I run with my dad and other parents in our school district. While SPARK club represents a broader learning framework and community effort, this particular project is designed and built by me with extensive use of AI tools for brainstorming and coding and little mentorship from my dad.

Identifying a Community Problem and Its Impact

The community problem I chose to address is **School Crossing Safety**. Through interviews with school administrators, I learned that crossing guard coverage is a persistent issue. The role is low-paying, physically demanding, and requires standing outdoors in extreme weather. As a result, it is difficult to staff consistently, even though it is critical for student safety.

SPARK was designed as a humanoid school crossing guard that can operate reliably and assist with traffic control during school hours. The primary beneficiaries are students, parents, school staff, and the surrounding community. By improving consistency and reliability in school crossings, SPARK helps reduce safety risks while allowing schools to focus limited resources elsewhere.

Alignment with National Priorities and American Innovation

SPARK directly supports the President's priorities around responsible artificial intelligence, community safety, STEM education, and workforce readiness. It is a hands-on learning platform that teaches students how to build and apply AI and robotics to real-world problems that improve public safety.

A core SPARK application is an AI-powered traffic guard that improves safety in school zones and neighborhoods through increased traffic awareness and pedestrian protection. Built on an extensible architecture, the same AI platform can scale from protecting individual schools to supporting neighborhood watch systems, city-level road safety through pothole detection, and even **national applications such as infrastructure and border protection**. This demonstrates how student-built, responsible AI can grow from local impact to solutions that strengthen communities, cities, and even the country as a whole.

SPARK was fully conceived and built in the United States, from hardware and electronics to software and AI model training. By avoiding pre-made imported kits and learning from first principles with simple, locally sourced materials, SPARK reflects American ingenuity while preparing students to responsibly collaborate with advanced AI technologies and contribute to a safer future workforce. **SPARK is proudly and truly made in the USA.**

Technical Design and System Architecture

SPARK is an edge-AI system that uses real-time computer vision to detect students at a school crossing and trigger coordinated physical actions through a modular, event-driven architecture. The system combines custom-built hardware, a custom-trained AI model, and downstream applications to demonstrate both technical understanding and functional reliability.



Building Blocks of the SPARK Platform

Hardware Design and Sensors

SPARK's design is inspired by real-world humanoid robots such as Boston Dynamics' Atlas and Tesla's Optimus, but reimaged using accessible materials. The robot is constructed from popsicle sticks and designed to stand at a mock school intersection. It features servo-driven arms and neck movement, along with a pan-tilt camera mounted on its helmet to provide continuous visual coverage of the environment.

In addition to the humanoid robot, the system includes smart traffic lights built using a [Raspberry Pi Pico W](#) and RGB LED strips, as well as a Raspberry Pi Pico W based RC car used to simulate vehicle behavior at an intersection.

AI Vision Pipeline

The AI system runs on an [NVIDIA Jetson](#) Nano, a compact edge AI computer, using the open-source [NVIDIA DeepStream](#) framework for real-time computer vision. A [custom-trained YOLO object detection model](#) analyzes live video from SPARK's camera to identify objects of interest. To prepare the training data, [Roboflow](#) was used to annotate images and manage dataset quality. For testing and validation, toys were used to simulate students waiting to cross.

When objects are detected, DeepStream converts detections into structured events that are published to an [MQTT](#) message broker (Mosquitto). This event-based design allows the AI system to remain focused on perception while enabling other components to react independently.

Application Logic and Physical Actions

AI-generated events are processed by downstream applications that act as the system's decision-making layer. The school crossing guard logic was implemented as a Python-based web application, developed using vibe coding. When the system detects three students waiting to cross, the application sends commands to SPARK to raise its stop sign and to the smart traffic lights, which transition from green to yellow and then red to allow students to cross safely.

To validate that AI-driven decisions could reliably control multiple devices, an RC car was added to the system. The RC car subscribes to the same MQTT messages and responds like a real vehicle by stopping at a red light and waiting until it is safe to move. This confirmed that the system could coordinate real-time actions across independent hardware components.

Extendability and Additional Applications

Beyond the core crossing guard use case, the same AI framework was extended to support a computer vision based **Student Attendance System**. This application uses the same detection and messaging pipeline, demonstrating that new AI-powered use cases can be added without modifying the core vision system.

AI Decision Flow

Camera input → AI object detection model → Event generation → Application logic → Real-world physical actions

Live video from SPARK's pan-tilt camera is analyzed in real time by the YOLO model running on the NVIDIA Jetson. Detected objects are converted into events by DeepStream and published through MQTT. Downstream applications subscribe to these events, apply safety and logic rules, and trigger coordinated actions such as raising the stop sign, changing traffic lights, stopping vehicles, or marking student attendance.

Conversational AI & Text-to-Speech

Our robot isn't just smart with its eyes. It is smart with its words too! Using LLM AI via the [ChatGPT API](#), it has a playful, engaging personality, can converse, and respond with humor to make interactions genuinely

fun. Combined with [OpenAI text-to-speech](#), the robot can announce, alert, and narrate actions in a lively voice. These AI features make the app interactive, entertaining, and almost alive.

Development Tools and AI Assistance

All supporting software, including the MicroPython code for the Raspberry Pi Pico W, was written with assistance from AI tools. [GitHub Copilot](#) with [Anthropic's Claude](#) was used for vibe coding across Python web applications and embedded code. ChatGPT was used for brainstorming, explanations, and mentorship. All final design decisions, testing, debugging, and validation were performed by me, Saanvi Pagadala.

Technical Stack Summary

- **AI Hardware:** NVIDIA Jetson Nano
- **AI Framework:** NVIDIA DeepStream
- **Model:** Custom-trained YOLO object detection, ChatGPT Conversational AI with text-to-speech
- **Dataset Annotation:** Roboflow
- **Messaging:** Mosquitto MQTT
- **Downstream Apps:** Python services and React interfaces
- **Embedded Control:** Raspberry Pi PicoW with MicroPython (Thonny)
- **Validation Devices:** Smart traffic lights and Raspberry Pi Pico W based RC car
- **Development Tools:** GitHub Copilot, Anthropic Claude, ChatGPT

Design Process, Iteration, and Development

Building SPARK was an iterative process that combined physical prototyping, AI development, and continuous testing. I started with simple popsicle-stick prototypes to refine the overall design and balance of the humanoid robot. Early versions were unstable and required several rebuilds to enhance stability and refine her charming, 'cute robot' appearance. One of the biggest mechanical challenges was choosing the right motors. I experimented with DC motors and several types of servo motors, but many lacked the precision, torque, or durability I needed for repeated motion. Through hands-on testing and comparison, I ultimately selected digital, full metal gear, 270-degree controllable servos, which provided smooth, reliable, and repeatable motion for the arms and neck.

On the AI side, my first object-detection model produced false positives and inconsistent results, especially when lighting conditions changed or objects were partially visible. I improved accuracy by retraining the model with better-labeled data, expanding the dataset, adjusting confidence thresholds, and testing in different lighting environments. I followed a vibe coding approach, using AI tools as mentors to rapidly prototype ideas, review code, identify bugs, and refine logic through multiple iterations while still validating every change through hands-on testing.

As part of my software development process, I also experimented with different AI models for vibe coding. I tested tools based on Grok, Gemini, and GPT to assist with generating and refining code. While these tools were helpful for explanations and high-level ideas, I found that the code they produced was not good and

required significant rework. After multiple trials, I discovered that Claude by Anthropic consistently generated clearer, more structured, and more reliable code. Through this experimentation, I learned that not all AI tools perform equally for the same task, and that selecting the right model through testing and comparison is an important part of responsible and effective AI development.

System integration was also a significant challenge. Synchronizing computer vision, messaging, and physical actions required multiple test cycles to resolve timing delays, dropped messages, and communication errors between the Jetson, the traffic lights, and the RC car. Early tests revealed latency issues and mismatched signals, which I addressed by refining message handling and adding safeguards in the application logic. Through repeated testing and debugging, I verified that detections consistently triggered the correct physical actions in real time across multiple devices.

Each setback led to measurable improvements, and through repeated iteration and refinement, SPARK evolved into a reliable, modular system that operates in real time and can be extended to support new use cases.

This project deepened my understanding of how AI systems work end to end, from data collection and model training to deployment and real-world decision-making. It also taught me the importance of responsible AI development, including thorough testing, validation, and designing systems that assist people in practical, meaningful ways rather than replacing them.

Process and Development Summary

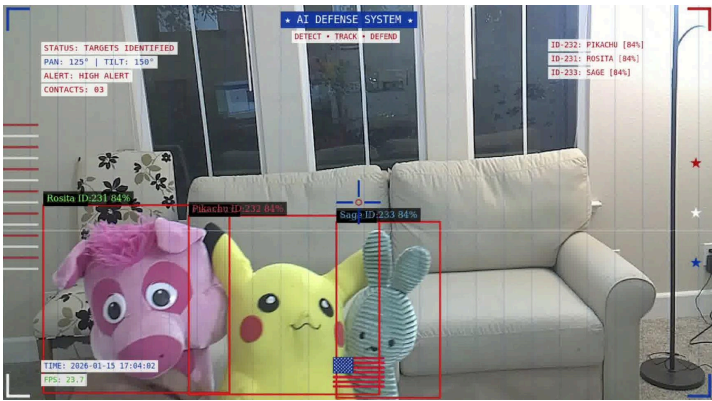
Area	Initial Challenge / Setback	Iteration & Testing	Improvement Achieved
Physical Design	Early popsicle-stick prototypes were unstable and lacked proper balance	Multiple rebuilds and structural adjustments	Improved stability, durability, and a refined, friendly humanoid appearance
Motor Selection	DC motors and early servos lacked precision, torque, or durability	Tested multiple motor types under repeated motion	Selected digital full metal gear 270-degree servos for smooth, reliable arm and neck movement
AI Object Detection	False positives and inconsistent detections, especially in varying lighting	Retrained model, improved labeling, adjusted confidence thresholds, tested in different conditions	Significantly improved detection accuracy and consistency
Vibe Coding Tools	Code generated by Grok, Gemini, and GPT required heavy rework	Compared outputs across models through multiple trials	Identified Claude by Anthropic as the most reliable for structured, usable code

System Integration	Timing delays, dropped messages, and mismatched signals across components	Multiple integration test cycles and debugging sessions	Reliable real-time coordination between vision, messaging, traffic lights, and RC car
End-to-End Validation	Early tests showed inconsistent physical responses to AI events	Repeated full-system testing across devices	Consistent, real-time AI-driven actions across multiple independent systems

AI Integration, Testing, and Validation

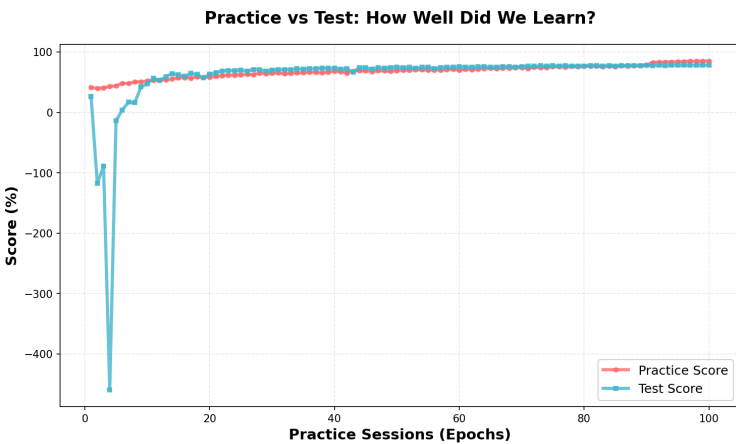
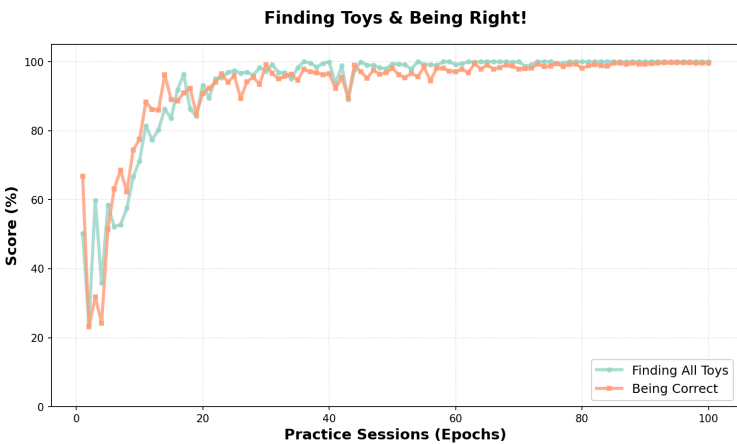
AI is the core intelligence behind SPARK. It is what allows the robot to perceive its environment, make decisions, and act safely. Instead of relying on fixed rules or timers, SPARK uses real-time computer vision to understand what is happening at the crosswalk.

My custom-trained YOLO object detection model can recognize children, using my favorite toys as safe stand-ins during development and testing. It runs entirely on an NVIDIA Jetson with open-source DeepStream, enabling fast, real-time detections.



Custom YOLO Model in Action

[On this page](#), you can explore the data labeling, training process, and performance results, with visual graphs tracking how the model improved over time. Below are two example graphs showing Accuracy and Validation curves from training.



Validation was a critical part of development. I tested the AI system under different lighting conditions, camera angles, distances, and object arrangements. I tracked when detections were missed, when false positives occurred, and how quickly the system responded. Through multiple iterations, I improved performance by refining training data, adjusting confidence thresholds, and requiring consistent detections across multiple frames before triggering actions.

The system was considered successful only when AI decisions consistently resulted in correct, observable behavior, raising the stop sign, changing traffic lights from yellow to red, and stopping the RC car reliably. Every AI decision produces an MQTT message that can be inspected and logged, making the system transparent, testable, and safe.

This approach ensures that SPARK uses AI responsibly, where perception, decision-making, and physical actions are tightly validated before operating in a real-world setting.

AI Validation Summary

Validation Area	What Was Tested	Method Used	Outcome
Object Detection Accuracy	Correct identification of students (simulated with toys)	Tested with different object positions, distances, and partial visibility	Accuracy improved after dataset refinement and retraining
Lighting Conditions	Model performance in bright, dim, and mixed lighting	Tested indoors under multiple lighting setups	Reduced false positives after threshold tuning
Camera Angles & Motion	Detection reliability during pan-tilt camera movement	Tested multiple camera angles and scan speeds	Stable detections after multi-frame confirmation
False Positives	Incorrect detections triggering actions	Logged and reviewed detection events	Eliminated unsafe triggers using confidence thresholds
Response Time	Time from detection to physical action	Measured action delays across test runs	Consistent real-time responses achieved
System Coordination	Synchronization between Jetson, lights, and RC car	End-to-end testing with MQTT messaging	Reliable multi-device coordination
Safety Scenarios	Behavior when fewer than 3 students are detected	Negative testing with no or partial detections	No unintended actions triggered
Transparency & Debugging	Ability to inspect AI decisions	Logged MQTT messages for each detection	System behavior fully observable and testable

Key Takeaway

By testing SPARK in many real-world conditions and learning from every mistake, I gained confidence that my AI system can make safe, reliable decisions. This experience showed me how careful testing and validation turn an idea into something that can truly help people, and it motivates me to keep improving and sharing this work with others.

Originality Through a Reusable AI Platform

What makes SPARK truly original is not just the humanoid robot itself, but the **flexible AI framework** behind it. Instead of building a single-purpose robot, I designed SPARK as a modular, reusable platform, where the same computer vision and messaging pipeline can power many different real-world applications.

After completing the school crossing guard use case, I extended the system using vibe coding to build a computer vision based **Student Attendance System** that detects faces and automatically marks students present. This addressed an additional need identified by my GATE teacher and reused the same core AI architecture without changing the underlying model or infrastructure.

Because of this design, SPARK can be adapted to solve many other community problems, such as *identifying food waste in the school cafeteria, monitoring safety in playgrounds, or supporting garden and hydroponic farm management*. Each new application builds on the same responsible AI pipeline, demonstrating how thoughtful system design can multiply impact.

By combining responsible AI, simple materials, and an open, extensible architecture, SPARK moves beyond a single project and becomes a platform that other students can learn from, improve, and deploy in their own schools and communities.

Sharing the Spark with the World

SPARK began with a simple question: Can AI and robotics solve a real problem in my school?

What it became is something much bigger - a humanoid robot, an extensible AI framework, and a way for students to learn by building technology from scratch.

The name SPARK has another meaning. A spark may start small, but it can ignite something much larger. My hope is that this project becomes a spark that grows into a global movement - one that inspires more girls to explore STEM, support one another, and create responsible AI together.

SPARK is proudly **100% made in the United States**, from the ideas and design, to the hardware, software, and AI systems, showcasing what student-led American ingenuity can achieve with creativity and persistence.

This is just the beginning. From school safety and attendance systems to future projects like drone swarms built from simple materials, SPARK is about turning curiosity into impact.

Thank you for the opportunity to share my project, and for supporting a future where AI is creative, responsible, and built by all of us.

Additional Notes and Media

The following resources are included to complement the project narrative:

- [Project Video Demonstration \(3.49 mins\)](#): A concise overview showing SPARK in action, highlighting its AI, robotics, and system capabilities.
- [Project Website \(Highly Recommended\)](#): Explore SPARK in depth, including its modules, system design, and interactive elements.
- [Teaching a Computer to Recognize Toys](#) with a Custom YOLO Model: Learn how I collected data, trained the YOLO model, and tracked its performance with graphs.
- [Video Series](#): Six short videos breaking down SPARK's components, AI workflow, and real-world applications.
- [How I Built the SPARK Platform](#): Step-by-step insights into SPARK's design, hardware, electronics, software, custom AI, and overall integration.

Certification of Originality

I certify that this SPARK platform project submission is my original work, created specifically for this [The Presidential AI Challenge 2025-26](#). The concept, design, implementation, testing, and presentation were developed solely for this submission and have not been submitted elsewhere.

All AI tools used, such as ChatGPT for idea exploration, writing reports, and guidance and GitHub Copilot with Anthropic's Claude for coding support, were applied responsibly, while all final design decisions, coding, testing, and refinements were completed by me.

All research materials, references, images, datasets, and diagrams have been properly credited, and no copyrighted material was used without permission.

I affirm that this submission accurately represents my work, learning process, and use of AI technologies.

Student Name: Saanvi Pagadala

Project Title: SPARK

Date: Jan 20, 2026